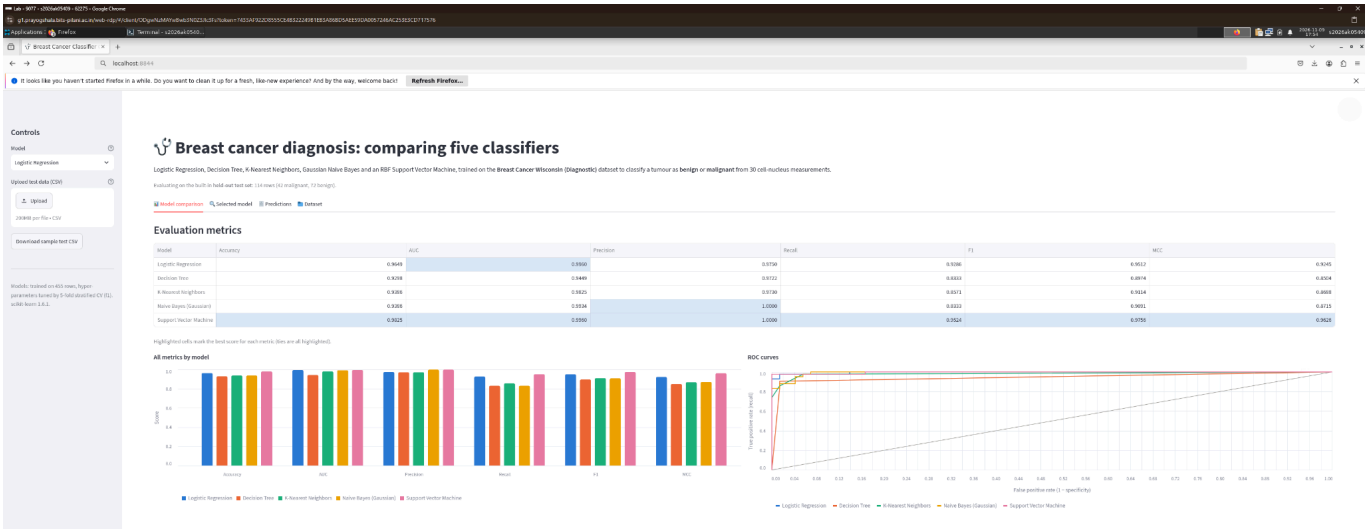
Assignment II - AIMLZG565
Machine Learning

Work Integrated Learning Programmes Division, M.Tech in AIML,
Semester 2026–27

NAME	ARKAPRAVA TRIPATHI
BITS ID	2026AK05409
PROJECT	Breast Cancer Diagnosis
GITHUB	https://github.com/IamARK21/BITS-ML-ASSIGNMENT-2
STREAMLIT	https://bits-ml-assignment-2-8bqdeq9vlwnrpmq7kccc45.streamlit.app/

The repository contains the project source, pinned dependencies, training and evaluation data, saved model pipelines, and README results.

BITS Virtual Lab Execution Screenshot



Breast Cancer Diagnosis: Comparing Five ML Classifiers

An end-to-end classification project. Five machine-learning models are trained on the same public dataset, evaluated with six metrics, and demonstrated in an interactive Streamlit web app.

a. Problem statement

Given 30 numeric measurements of cell nuclei taken from a digitised image of a fine-needle aspirate (FNA) of a breast mass, predict whether the tumour is **malignant (M)** or **benign (B)**. This is a **binary classification** problem. The positive class is *malignant*, so recall measures how many cancers the model catches.

b. Dataset description

Name	Breast Cancer Wisconsin (Diagnostic)
Source	UCI Machine Learning Repository: https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic (also on Kaggle as *uciml/breast-cancer-wisconsin-data*)
Instances	569 (requirement: at least 500)
Features	30 numeric, real-valued (requirement: at least 12)
Target	`diagnosis`: M = malignant (212, 37.3%), B = benign (357, 62.7%)
Missing values	None
File	data/breast_cancer.csv (raw `wdbc.data` with a header row added)

Ten properties are measured for each nucleus: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension. Each is summarised three ways across the nuclei in the image: the **mean**, the **standard error** (`_se`) and the **worst** value (mean of the three largest). That gives 30 features such as `radius_mean`, `radius_se` and `radius_worst`. The `id` column is an identifier and is not used for training.

Train/test split. 80/20 stratified split with `random_state=42`: 455 training rows and 114 test rows (72 benign, 42 malignant). The test rows are saved as data/test_data.csv for uploading to the app.

c. GitHub repository link

- **Repository:** https://github.com/IamARK21/BITS-ML-ASSIGNMENT-2
- **Live Streamlit app:** https://bits-ml-assignment-2-8bqdeq9vlwnrpmq7kccc45.streamlit.app/

d. Models used and evaluation

i. Models used and comparison table

All five models are trained on the same training split. Each one's hyper-parameters are tuned with 5-fold stratified cross-validation (scored on F1) on the **training data only**. Each model is then evaluated once on the untouched test set.

Model	Preprocessing	Hyper-parameters searched
Logistic Regression	StandardScaler	`C` ∈ {0.01, 0.1, 1, 10, 100}
Decision Tree Classifier	none (scale-invariant)	`criterion`, `max_depth` ∈ {3, 4, 5, 6, 8, None}, `min_samples_leaf` ∈ {1, 2, 4, 8}
K-Nearest Neighbor Classifier (KNN)	StandardScaler	`n_neighbors` ∈ {3, …, 21}, `weights` ∈ {uniform, distance}
Naive Bayes Classifier (Gaussian)	none (scale-invariant)	`var_smoothing` ∈ {1e-11, …, 1e-5}
Support Vector Machine (RBF kernel)	StandardScaler	`C` ∈ {0.1, 1, 10, 100}, `gamma` ∈ {scale, 0.1, 0.01, 0.001}

Scaling matters here because raw feature ranges differ by about six orders of magnitude (`area_worst` reaches ~4,000, `fractal_dimension_se` is ~0.003). Distance- and margin-based models (KNN, SVM) and regularised Logistic Regression would otherwise be dominated by the large-valued features. Gaussian Naive Bayes models each feature separately, and decision-tree splits are threshold-based, so neither is affected by per-feature scaling.

Comparison table (test set, n = 114, positive class = malignant):

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
Decision Tree	0.9298	0.9449	0.9722	0.8333	0.8974	0.8504
KNN	0.9386	0.9825	0.9730	0.8571	0.9114	0.8688
Naive Bayes	0.9386	0.9934	1.0000	0.8333	0.9091	0.8715
Support Vector Machine	0.9825	0.9960	1.0000	0.9524	0.9756	0.9626

Metric definitions:

- **Accuracy**: share of all predictions that are correct.
- **AUC**: area under the ROC curve, computed from predicted probabilities. Threshold-independent.
- **Precision**: of the tumours predicted malignant, the share that are malignant.
- **Recall**: of the malignant tumours, the share that were detected.
- **F1**: harmonic mean of precision and recall.
- **MCC**: Matthews correlation coefficient. Uses all four confusion-matrix cells and is robust to the 63/37 class imbalance.

Tuned hyper-parameters and test-set confusion matrices. FN = malignant tumour predicted benign (missed cancer). FP = benign tumour predicted malignant (false alarm).

ML Model Name	Tuned hyper-parameters	CV F1 (train)	TN	FP	FN	TP	Errors
Logistic Regression	`C=1`	0.9643	71	1	3	39	4
Decision Tree	`criterion=entropy, max_depth=6, min_samples_leaf=1`	0.9345	71	1	7	35	8
KNN	`n_neighbors=3, weights=uniform`	0.9668	71	1	6	36	7
Naive Bayes	`var_smoothing=1e-9`	0.9208	72	0	7	35	7
Support Vector Machine	`C=10, gamma=0.01` (RBF)	0.9699	72	0	2	40	2

ii. Observations on the performance of each model

ML Model Name	Observation about model performance
Logistic Regression	Second best overall (accuracy 0.9649, MCC 0.9245). It shares the highest AUC (0.9960) with the SVM, so it ranks malignant cases above benign ones just as well. It made 4 errors: 3 missed malignancies and 1 false alarm. Tuning kept the default regularisation (`C=1`). A linear model doing this well shows that, once standardised, the two classes are almost linearly separable.
Decision Tree	The weakest model: lowest or tied-lowest on all six metrics (accuracy 0.9298, F1 0.8974, MCC 0.8504). It missed 7 of the 42 malignant tumours (recall 0.8333). Its AUC (0.9449) is well below the others because a deep tree with near-pure leaves (`max_depth=6`, `min_samples_leaf=1`) outputs probabilities close to 0 or 1, which rank cases only coarsely. Its axis-parallel splits can only approximate the class boundary in steps, and a single deep tree has high variance.
KNN	Accuracy 0.9386 and MCC 0.8688, with 6 missed malignancies and 1 false alarm. Cross-validation chose `k=3`, which had the second-best CV F1 (0.9668), but test F1 fell to 0.9114. That is the largest CV-to-test drop of any model, and typical of a small, high-variance neighbourhood. With k=3 the predicted probability can only be 0, ⅓, ⅔ or 1, which caps AUC (0.9825). Distances are also dominated by redundant features: radius, perimeter and area are almost perfectly correlated (r = 0.97–0.998), so tumour size is effectively counted several times.
Naive Bayes	Perfect precision (1.0000, no false alarms) but recall of only 0.8333: 7 of 42 malignant tumours missed, tied for worst. Yet its AUC is high (0.9934), so it ranks cases well but its 0.5 decision threshold is badly placed. A lower threshold would catch more cancers. This matches its core assumption failing on this data: the size features are strongly correlated (r ≈ 0.97–0.998) but are treated as independent evidence, which distorts the probabilities. It also had the lowest CV F1 (0.9208).
Support Vector Machine	Best model on every metric: accuracy 0.9825, recall 0.9524, F1 0.9756, MCC 0.9626. It ties Logistic Regression on AUC (0.9960) and Naive Bayes on precision (1.0000). Only 2 errors on the test set, both missed malignancies, and no false alarms. It also had the best CV F1 (0.9699), so its lead holds across folds and isn't an artefact of one split. The tuned RBF kernel (`C=10`, `gamma=0.01`, about a third of scikit-learn's default gamma for 30 standardised features) gives a smooth maximum-margin boundary that suits these well-separated, scaled features.
Overall, winner for this dataset	**Support Vector Machine.** It is best or tied-best on all six metrics and has the best cross-validated F1. Ranking by MCC: SVM (0.9626) > Logistic Regression (0.9245) > Naive Bayes (0.8715) > KNN (0.8688) > Decision Tree (0.8504). Almost every error across the five models is a missed cancer (25 false negatives against 3 false positives), and precision is at least 0.97 everywhere. So recall (0.83–0.95) is what separates the models, and for a diagnostic aid a missed malignancy costs far more than a false alarm. The SVM has the highest recall (0.9524). The test set is small (114 rows, 42 malignant): one error moves accuracy by 0.9 points and one missed cancer moves recall by 2.4 points. KNN (7 errors), Naive Bayes (7) and the Decision Tree (8) are within one misclassification of each other, so their relative order is not meaningful.

Streamlit app

`app.py` loads the **saved** models from `model/*.pkl` and only predicts with them. It never trains or retrains a model, on the uploaded test data or otherwise. If the saved models can't be loaded, it shows an error instead.

Requirement	Where in the app
a. Dataset upload option (CSV)	Sidebar **Upload test data (CSV)**. The file must contain all 30 feature columns and the `diagnosis` target column (M/B or 1/0). Files missing either are rejected with a clear message. Rows with missing or non-numeric values are dropped and reported. The Kaggle copy of the dataset can be uploaded unchanged.
b. Model selection dropdown	Sidebar **Model** dropdown with all five models
c. Display of evaluation metrics	**Model comparison** tab: all six metrics for all five models on the test data, best per metric highlighted, plus a grouped bar chart and ROC curves. **Selected model** tab: the six metrics for the chosen model.
d. Confusion matrix or classification report	**Selected model** tab shows both

If no file is uploaded, the app evaluates the held-out test set. That set can also be downloaded from the sidebar (**Download sample test CSV**) and uploaded back. Two more tabs show per-row **Predictions** (probability of malignancy, a misclassified-only filter, CSV download) and the **Dataset** description.

`tests/smoke_test_app.py` checks this behaviour headlessly with Streamlit's `AppTest`. The app runs for every model. On the built-in test set and on uploaded CSVs, the metrics shown match `model/metrics.csv`. Invalid uploads are rejected. No model is ever fitted: `fit` is disabled for the whole test.

Project structure

```
ML_ASSIGNMENT_2/
├── app.py                    # Streamlit web application
├── colab_training.ipynb      # runs training + app test on Google Colab
├── requirements.txt          # scikit-learn pinned to the version that saved the models
├── README.md
├── tests/
│   └── smoke_test_app.py     # headless Streamlit AppTest of app.py
├── data/
│   ├── breast_cancer.csv     # full dataset (569 × 32)
│   └── test_data.csv         # held-out test split, generated by training
└── model/
    ├── common.py             # feature list, label encoding, upload validation, metrics
    ├── train_models.py       # trains, tunes, evaluates and saves all five models
    ├── *.pkl                 # fitted pipelines, generated by training
    ├── metrics.csv           # test-set metrics, generated by training
    └── metadata.json         # split sizes, tuned hyper-parameters, versions
```